Project Design Phase-II

Technology Stack (Architecture & Stack)

Date	09 March 2026
Team ID	698c67ecb31ff1dfca225748
Project Name	PlantCare AI: Advanced Plant Disease Detection Using Transfer Learning
Team Members	Akshat Bhati, Subhadipta Sarkar, Haridharshan B, Madan M
Maximum Marks	4 Marks

Technical Architecture Overview

PlantCare AI is built on a three-tier architecture: (1) Presentation Layer — four HTML/CSS/Jinja2 web pages, (2) Application Layer — Flask Python server handling routing, sessions, and file management, and (3) Intelligence Layer — MobileNetV2 Keras model performing 38-class plant disease inference. There is no external database; state is managed via Flask sessions and the filesystem.

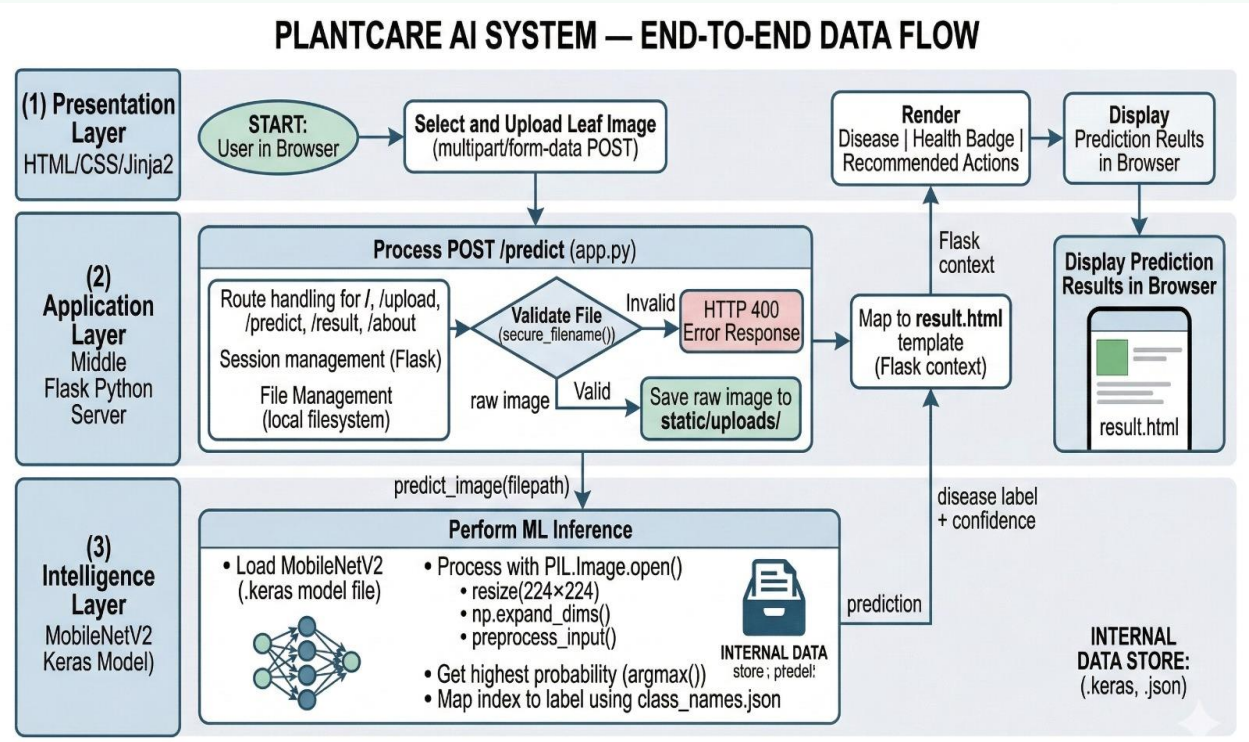

Diagram 3: Intelligence Layer (Keras Model)

Table 1: Components & Technologies

S.No	Component	Description	Technology
1	User Interface	Four-page responsive web application: Home (hero, features, stats), About (mission, how-it-works), Upload (drag-and-drop zone), Result (disease badge, actions). Served by Flask Jinja2.	HTML5, CSS3, JavaScript, Jinja2 Templating
2	Application Logic-1	Flask web server: HTTP routing, session management, file upload/download, error handling, redirect logic across all 5 routes	Flask 3.x (Python) — Werkzeug WSGI middleware
3	Application Logic-2	Image preprocessing pipeline: PIL load, resize to 224×224, convert to float32 NumPy array, expand dims, apply MobileNetV2 preprocess_input() normalization	Python 3.x + Pillow (PIL) + NumPy
4	Application Logic-3	Transfer learning training pipeline: MobileNetV2 base, freeze/unfreeze 10 layers, augmented ImageDataGenerator, model.fit() with callbacks, training curve plots	TensorFlow 2.17+ / Keras — Google Colab GPU (T4)
5	Model Storage	Trained Keras model stored on local filesystem; class name list stored as JSON; both loaded once at Flask startup	Local Filesystem (.keras + class_names.json)
6	File Storage	Temporary storage for user-uploaded leaf images during active inference session; images retained for result display	Local Filesystem — static/uploads/ directory
7	Dataset / External Source	New Plant Diseases Dataset — 87,000+ augmented JPEGs, 38 classes, downloaded programmatically in Google Colab	Kaggle API (kaggle.json credentials) — Dataset: vipoooool/new-plant-diseases-dataset
8	Machine Learning Model	MobileNetV2 CNN pretrained on ImageNet-1000; last 10 layers unfrozen for fine-tuning; custom head: GlobalAvgPool2D → Dropout(0.35) → Dense(256) → Dropout(0.25) → Dense(38, softmax)	TensorFlow/Keras — MobileNetV2 Transfer Learning
9	Infrastructure (Local)	Flask built-in development server for local testing and iterative development	Python Flask dev server — localhost:5000
10	Infrastructure (Cloud/Prod)	Production deployment: WSGI server behind reverse proxy; container-ready architecture for cloud platforms	Gunicorn + Nginx / Docker / Heroku / Google Cloud Run

Table 2: Application Characteristics

S.No	Characteristic	Description	Technology
1	Open-Source Frameworks	TensorFlow/Keras (Apache 2.0), Flask (BSD-3), NumPy (BSD), Pillow (HPND), Werkzeug (BSD), Matplotlib (PSF), scikit-learn (BSD-3), Seaborn (BSD)	All Apache 2.0 / BSD licensed — fully open-source stack
2	Security Implementations	Werkzeug secure_filename() sanitizes all upload filenames preventing path traversal; file extension whitelist (.jpg .jpeg .png .gif); Flask session encrypted with secret key; try/except error boundaries; no raw SQL (no SQL injection surface)	Werkzeug file security, Flask session encryption, input validation, OWASP best practices
3	Scalable Architecture	3-tier separation (Presentation / Application / Intelligence); model loaded once at startup (not per-request); stateless Flask routes; Docker-containerizable; API endpoints separable for microservice evolution	3-tier architecture + Flask + Gunicorn multi-worker + Docker
4	Availability	Local: single Flask instance (dev); Production: Gunicorn multi-worker WSGI + Nginx load balancer; cloud platforms provide managed uptime SLA; static file caching via Nginx reduces Flask load	Gunicorn (multi-worker) + Nginx reverse proxy + Cloud PaaS (99.9% SLA)
5	Performance	MobileNetV2 inference under 1.5s on CPU (3.4M params vs VGG16 138M — 40× lighter); model loaded once at startup eliminating cold-start overhead; Nginx serves static files bypassing Python; Batch size 32 maximizes GPU throughput during training	MobileNetV2 depthwise separable convolutions + Nginx static serving + Gunicorn workers

References: https://c4model.com/ | https://aws.amazon.com/architecture | https://developer.ibm.com/patterns/